IC-253 (DSA) Assignment 4 Report

Aryan Sisodiya

April 23, 2026

1 Task 1: Smart City Road Network Optimizer

1.1 Problem Description

This task models a city road network as a weighted graph with construction costs and travel times. Two planning objectives are solved using the same graph:

- Minimum-cost infrastructure design using Minimum Spanning Tree.
- Minimum travel-time routing using shortest path computation.

Additional analysis compares both networks and identifies critical roads whose removal increases MST cost.

1.2 Approach

The implementation uses:

- Adjacency-list graph representation.
- Edges are sorted in-place by quicksort for Kruskal's algorithm.
- Kruskal's algorithm with Disjoint Set Union for MST.
- Dijkstra's algorithm using min-heap for shortest path.
- DFS over MST edges for comparing MST route travel time.
- Edge-removal sensitivity analysis for critical roads.

1.3 Graph Input Validation Policy

The graph validation policy enforced by `validateGraph()` is:

- Node IDs must lie in range $[0, N-1]$, where N is total Vertices in graph.
- Self-loops are forbidden.
- Duplicate undirected roads are rejected.
- Construction costs and travel times must be non-negative.
- Disconnected graphs are reported during MST construction.
- Invalid source or destination nodes are rejected at query stage.

Rejected inputs print a specific failure reason.

1.4 Functions and Implementation

validateGraph(edges,N)
Checks graph validity according to the policy.

buildMST()
Constructs the MST using Kruskal and prints accept/reject decisions.

shortestPath(S,T)
Computes and prints optimal travel-time path.

compareNetworks()
Compares MST path and globally shortest path.

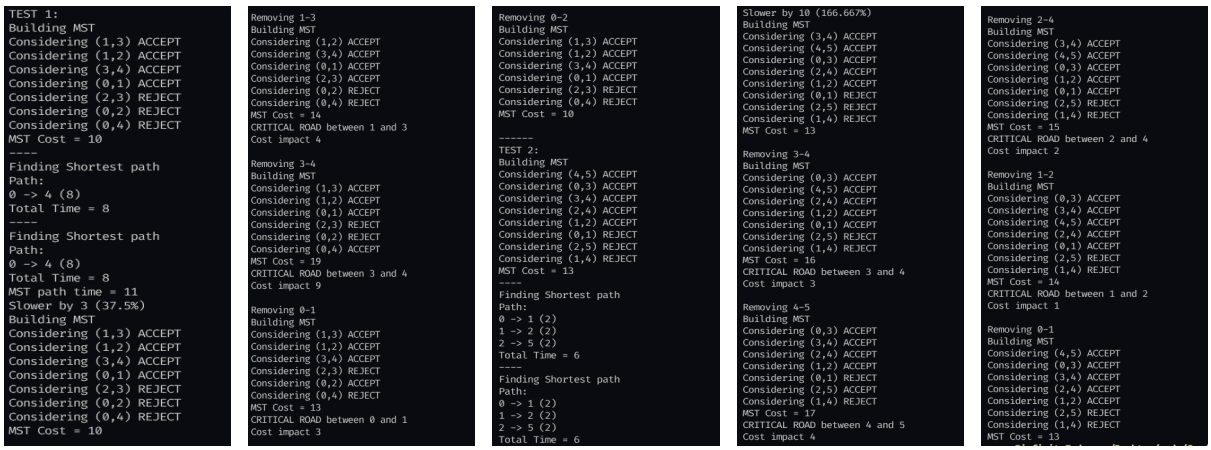
criticalRoads()
Finds roads whose removal increases MST cost.

1.5 Time and Space Complexity Analysis

Operation	Time Complexity	Space Complexity
validateGraph()	$O(M^2)$	$O(1)$
buildMST()	$O(M \log M)$	$O(N)$
shortestPath()	$O((N+M)\log N)$	$O(N+M)$
compareNetworks()	$O((N+M)\log N)$	$O(N)$
criticalRoads()	$O((N-1)M \log M)$	$O(N)$

where N is number of intersections and M is number of roads.

1.6 Test Case Demonstrations



```
TEST 1:
Building MST
Considering (1,3) ACCEPT
Considering (1,2) ACCEPT
Considering (3,4) ACCEPT
Considering (0,1) ACCEPT
Considering (2,3) REJECT
Considering (0,2) REJECT
Considering (0,4) REJECT
MST Cost = 10
----
Finding Shortest path
Path:
0 -> 4 (8)
Total Time = 8
----
Finding Shortest path
Path:
0 -> 4 (8)
Total Time = 8
MST path time = 11
Slower by 3 (37.5%)
Building MST
Considering (1,3) ACCEPT
Considering (1,2) ACCEPT
Considering (3,4) ACCEPT
Considering (0,1) ACCEPT
Considering (2,3) REJECT
Considering (0,2) REJECT
Considering (0,4) REJECT
MST Cost = 10
```

```
Removing 1-3
Building MST
Considering (1,2) ACCEPT
Considering (3,4) ACCEPT
Considering (0,1) ACCEPT
Considering (2,3) ACCEPT
Considering (0,2) REJECT
Considering (0,4) REJECT
MST Cost = 14
CRITICAL ROAD between 1 and 3
Cost impact 4

Removing 3-4
Building MST
Considering (1,3) ACCEPT
Considering (1,2) ACCEPT
Considering (0,1) ACCEPT
Considering (2,3) REJECT
Considering (0,2) REJECT
Considering (0,4) ACCEPT
MST Cost = 19
CRITICAL ROAD between 3 and 4
Cost impact 9

Removing 0-1
Building MST
Considering (1,3) ACCEPT
Considering (1,2) ACCEPT
Considering (3,4) ACCEPT
Considering (2,3) REJECT
Considering (0,2) ACCEPT
Considering (0,4) REJECT
MST Cost = 13
CRITICAL ROAD between 0 and 1
Cost impact 3
```

```
Removing 0-2
Building MST
Considering (1,3) ACCEPT
Considering (1,2) ACCEPT
Considering (3,4) ACCEPT
Considering (0,1) ACCEPT
Considering (2,3) REJECT
Considering (0,4) REJECT
MST Cost = 10

------
TEST 2:
Building MST
Considering (4,5) ACCEPT
Considering (0,3) ACCEPT
Considering (3,4) ACCEPT
Considering (2,4) ACCEPT
Considering (1,2) ACCEPT
Considering (0,1) REJECT
Considering (2,5) REJECT
Considering (1,4) REJECT
MST Cost = 13
----
Finding Shortest path
Path:
0 -> 1 (2)
1 -> 2 (2)
2 -> 5 (2)
Total Time = 6
----
Finding Shortest path
Path:
0 -> 1 (2)
1 -> 2 (2)
2 -> 5 (2)
Total Time = 6
```

```
Slower by 10 (166.667%)
Building MST
Considering (3,4) ACCEPT
Considering (4,5) ACCEPT
Considering (0,3) ACCEPT
Considering (2,4) ACCEPT
Considering (1,2) ACCEPT
Considering (0,1) REJECT
Considering (2,5) REJECT
Considering (1,4) REJECT
MST Cost = 13

Removing 3-4
Building MST
Considering (0,3) ACCEPT
Considering (4,5) ACCEPT
Considering (2,4) ACCEPT
Considering (1,2) ACCEPT
Considering (0,1) ACCEPT
Considering (2,5) REJECT
Considering (1,4) REJECT
MST Cost = 16
CRITICAL ROAD between 3 and 4
Cost impact 3

Removing 4-5
Building MST
Considering (0,3) ACCEPT
Considering (3,4) ACCEPT
Considering (2,4) ACCEPT
Considering (1,2) ACCEPT
Considering (0,1) REJECT
Considering (2,5) ACCEPT
Considering (1,4) REJECT
MST Cost = 17
CRITICAL ROAD between 4 and 5
Cost impact 4
```

```
Removing 2-4
Building MST
Considering (3,4) ACCEPT
Considering (4,5) ACCEPT
Considering (0,3) ACCEPT
Considering (1,2) ACCEPT
Considering (0,1) ACCEPT
Considering (2,5) REJECT
Considering (1,4) REJECT
MST Cost = 15
CRITICAL ROAD between 2 and 4
Cost impact 2

Removing 1-2
Building MST
Considering (0,3) ACCEPT
Considering (3,4) ACCEPT
Considering (4,5) ACCEPT
Considering (2,4) ACCEPT
Considering (0,1) ACCEPT
Considering (2,5) REJECT
Considering (1,4) REJECT
MST Cost = 14
CRITICAL ROAD between 1 and 2
Cost impact 1

Removing 0-1
Building MST
Considering (4,5) ACCEPT
Considering (0,3) ACCEPT
Considering (3,4) ACCEPT
Considering (2,4) ACCEPT
Considering (1,2) ACCEPT
Considering (2,5) REJECT
Considering (1,4) REJECT
MST Cost = 13
```

Figure 1: Task 1 – Test Case Outputs

2 Task 2: Logistics Dispatch Centre

2.1 Problem Description

This task models a disaster-relief request dispatch system supporting both dynamic real-time dispatch and in-place record sorting.

Requests contain:

- Unique request ID
- Priority score

- Submission timestamp

A max-heap supports live dispatch, while randomized in-place quicksort supports reporting and accounting views.

2.2 Approach

The implementation uses:

- Array-based max heap for dispatch queue.
- Position map for efficient priority updates.
- Heapify-up and heapify-down restoration.
- Randomized quicksort for in-place record sorting.
- Backup-and-restore strategy for non-destructive top-k extraction.

2.3 Data Handling Policy

The request handling policy enforced by `validateInput()` and updates is:

- Duplicate request IDs are rejected.
- Valid priority range is 0 to 100.
- Negative IDs are invalid.
- Unknown IDs in update requests are rejected.
- Out-of-range priority updates are rejected.
- Request IDs are assumed bounded by implementation constant `MAXN`.

All invalid operations print explicit reasons.

2.4 Functions and Implementation

validateInput(requests)
Validates request list.

buildStructure(requests)
Constructs max heap and prints swaps.

dispatchNext()
Dispatches highest-priority request.

updatePriority(id,p)
Updates priority and restores heap invariant.

sortRecords(records,key)
Performs randomized in-place quicksort.

topKRequests(k)
Returns top k requests without modifying live structure.

2.5 Time and Space Complexity Analysis

Operation	Time Complexity	Space Complexity
validateInput()	$O(n)$	$O(n)$
buildStructure()	$O(n)$	$O(1)$
dispatchNext()	$O(\log n)$	$O(1)$
updatePriority()	$O(\log n)$	$O(1)$
sortRecords()	$O(n \log n)$ average	$O(\log n)$
topKRequests()	$O(k \log n)$	$O(n)$

2.6 Test Case Demonstrations

```
TEST CASE 1
Building Max Heap
Swap 12 and 16
Swap 11 and 13
Swap 10 and 13
Swap 10 and 11
Heap state (ID, PRIORITY):
[13,90] [11,80] [16,70] [10,55] [14,60] [15,50] [12,35]

Dispatch Next
Dispatching ID: 13, priority: 90
Swap 12 and 11
Swap 12 and 14
Heap state (ID, PRIORITY):
[11,80] [14,60] [16,70] [10,55] [12,35] [15,50]

Increase Priority
Updating ID 12: 35 -> 95
Current Heap State:
Moving upward
Swap 14 and 12
Swap 11 and 12
Heap state (ID, PRIORITY):
[12,95] [11,80] [16,70] [10,55] [14,60] [15,50]

Decrease Priority
Updating ID 14: 60 -> 20
Current Heap State:
Moving downward
Heap state (ID, PRIORITY):
[12,95] [11,80] [16,70] [10,55] [14,20] [15,50]
```

```
Top K Requests
Top 3 requests:
Dispatching ID: 12, priority: 95
Swap 15 and 11
Swap 15 and 10
Heap state (ID, PRIORITY):
[11,80] [10,55] [16,70] [15,50] [14,20]
ID: 12, Priority: 95
Dispatching ID: 11, priority: 80
Swap 14 and 16
Heap state (ID, PRIORITY):
[16,70] [10,55] [14,20] [15,50]
ID: 11, Priority: 80
Dispatching ID: 16, priority: 70
Swap 15 and 10
Heap state (ID, PRIORITY):
[10,55] [15,50] [14,20]
ID: 16, Priority: 70

Sort By Priority
Pivot: 11
Partition boundaries: [0,6]
Pivot: 15
Partition boundaries: [0,4]
Pivot: 10
Partition boundaries: [2,4]
Pivot: 16
Partition boundaries: [3,4]
Records (ID, PRIORITY, TIME):
(12,35,5) (15,50,8) (10,55,7) (14,60,4) (16,70,3) (11,80,2) (13,90,1)
```

```
Sort By Timestamp
Pivot: 11
Partition boundaries: [0,6]
Pivot: 12
Partition boundaries: [2,6]
Pivot: 16
Partition boundaries: [2,3]
Pivot: 10
Partition boundaries: [5,6]
Records (ID, PRIORITY, TIME):
(13,90,1) (11,80,2) (16,70,3) (14,60,4) (12,35,5) (10,55,7) (15,50,8)

-------
TEST CASE 2
Building Max Heap
Swap 24 and 28
Swap 23 and 27
Swap 22 and 25
Swap 21 and 27
Swap 21 and 23
Heap state (ID, PRIORITY):
[27,95] [25,75] [23,85] [28,60] [22,65] [26,50] [21,40] [24,30]

Dispatch Two Requests
Dispatching ID: 27, priority: 95
Swap 24 and 23
Swap 24 and 26
Heap state (ID, PRIORITY):
[23,85] [25,75] [26,50] [28,60] [22,65] [24,30] [21,40]
Dispatching ID: 23, priority: 85
```

```
Swap 21 and 25
Swap 21 and 22
Heap state (ID, PRIORITY):
[25,75] [22,65] [26,50] [28,60] [21,40] [24,30]

Priority Update
Updating ID 24: 30 -> 88
Current Heap State:
Moving upward
Swap 26 and 24
Swap 25 and 24
Heap state (ID, PRIORITY):
[24,88] [22,65] [25,75] [28,60] [21,40] [26,50]

Top 4 Requests
Top 4 requests:
Dispatching ID: 24, priority: 88
Swap 26 and 25
Heap state (ID, PRIORITY):
[25,75] [22,65] [26,50] [28,60] [21,40]
ID: 24, Priority: 88
Dispatching ID: 25, priority: 75
Swap 21 and 22
Swap 21 and 28
Heap state (ID, PRIORITY):
[22,65] [28,60] [26,50] [21,40]
ID: 25, Priority: 75
Dispatching ID: 22, priority: 65
Swap 21 and 28
```

```
Heap state (ID, PRIORITY):
[28,60] [21,40] [26,50]
ID: 22, Priority: 65
Dispatching ID: 28, priority: 60
Heap state (ID, PRIORITY):
[26,50] [21,40]
ID: 28, Priority: 60

Sort By Priority
Pivot: 23
Partition boundaries: [0,7]
Pivot: 28
Partition boundaries: [0,5]
Pivot: 24
Partition boundaries: [0,2]
Pivot: 21
Partition boundaries: [1,2]
Pivot: 25
Partition boundaries: [4,5]
Records (ID, PRIORITY, TIME):
(24,30,8) (21,40,9) (26,50,7) (28,60,6) (22,65,4) (25,75,3) (23,85,1) (27,95,2)

Sort By Timestamp
Pivot: 25
Partition boundaries: [0,7]
Pivot: 23
Partition boundaries: [0,1]
Pivot: 24
Partition boundaries: [3,7]
Pivot: 26
Partition boundaries: [3,5]
Pivot: 22
Partition boundaries: [3,4]
Records (ID, PRIORITY, TIME):
(23,85,1) (27,95,2) (25,75,3) (22,65,4) (28,60,6) (26,50,7) (24,30,8) (21,40,9)
```

Figure 2: Task 2 – Test Case Outputs